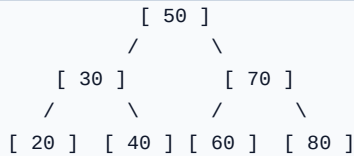


# Binary Search Tree (BST) & Operations -- Data Structures

**Definition:** A Binary Search Tree (BST) is a binary tree with the ordering property: for every node X, all keys in X's left subtree are strictly less than X.key, and all keys in X's right subtree are strictly greater than X.key.

## 1. Detailed Technical Explanation



### 1. Search Operation in BST

\* Compare search key K with root->data:

\* If K

### 2. Insertion Operation

\* Traverse tree following BST property until reaching a NULL pointer, then insert new node as a leaf.

### 3. Deletion Operation in BST (Three Cases):

1. Case 1: Node is a Leaf (0 Children): Simply delete the node and set parent's pointer to NULL.

2. Ca

## 2. Complete Executable C Implementation of BST

```

#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *left, *right;
};

struct Node* createNode(int key) {
    struct Node* n = (struct Node*)malloc(sizeof(struct Node));
    n->data = key;
    n->left = n->right = NULL;
    return n;
}

// 1. BST Insert
struct Node* insert(struct Node* node, int key) {
    if (node == NULL) return createNode(key);
    if (key < node->data)
        node->left = insert(node->left, key);
    else if (key > node->data)
        node->right = insert(node->right, key);
    return node;
}
  
```

```
// Find minimum node (Inorder Successor helper)
struct Node* findMin(struct Node* node) {
    struct Node* current = node;
    while (current && current->left != NULL)
        current = current->left;
    return current;
}

// 2. BST Delete
struct Node* deleteNode(struct Node* root, int key) {
    if (root == NULL) return root;

    if (key < root->data)
        root->left = deleteNode(root->left, key);
    else if (key > root->data)
        root->right = deleteNode(root->right, key);
    else {
        // Case 1 & 2: 0 or 1 child
        if (root->left == NULL) {
            struct Node* temp = root->right;
            free(root);
            return temp;
        } else if (root->right == NULL) {
            struct Node* temp = root->left;
            free(root);
            return temp;
        }
        // Case 3: 2 children
        struct Node* temp = findMin(root->right); // Inorder successor
        root->data = temp->data;
        root->right = deleteNode(root->right, temp->data);
    }
    return root;
}

void inorder(struct Node* root) {
    if (root != NULL) {
        inorder(root->left);
        printf("%d ", root->data);
        inorder(root->right);
    }
}
```

### 3. Time and Space Complexities

| Operation            | Average Case | Worst Case (Skewed) |
|----------------------|--------------|---------------------|
| <b>**Search**</b>    | $O(\log N)$  | $O(N)$              |
| <b>**Insertion**</b> | $O(\log N)$  | $O(N)$              |
| <b>**Deletion**</b>  | $O(\log N)$  | $O(N)$              |
| <b>**Space**</b>     | $O(H)$       | $O(N)$              |

### 4. Quick Recall Flow

BST Property: Left < Root < Right -> Search/Insert via Binary Decision -> Delete (0 child: drop, 1 child: bypass, 2 children: inorder successor)